Intelligent Dead Reckoning

AI/ML-based navigation for GPS-denied tunnels

Interview questions & answers

M.Tech 1st Semester · Machine Learning Project

93 questions · 18 parts · glossary of technical terms

How to read this deck

This is written as an interview. The interviewer asks, and the person who built the project answers. The answers are written for someone who is NOT a technical expert, but every technical word is kept and explained in brackets the first time it appears. A glossary of all terms is at the end.

All numbers come from the project's own results (results/metrics.md, PROGRESS.md). “**m**” = metres, “**m/s**” = metres per second (1 m/s = 3.6 km/h).

What this deck covers

Part 1	The original 14 questions	Q1–Q14
Part 2	The big picture: what we built and why	Q15–Q20
Part 3	The data	Q21–Q25
Part 4	Problems in the data and how we fixed them	Q26–Q29
Part 5	Calibration: making phone sensors usable	Q30–Q34
Part 6	Baselines: the simple methods we had to beat	Q35–Q37
Part 7	The machine-learning speed model	Q38–Q42
Part 8	Using the model inside a tunnel (anchored ML)	Q43–Q45
Part 9	Direction (heading) and the 2-D path	Q46–Q49
Part 10	Map matching: keeping the car on real roads	Q50–Q54
Part 11	Ideas that did not work (and why we kept them in the report)	Q55–Q58
Part 12	GNSS + INS fusion (the Kalman filter)	Q59–Q65
Part 13	Making it run on a phone	Q66–Q69
Part 14	The edge engine, the app and route verification	Q70–Q78
Part 15	Honest evaluation: how we avoided cheating	Q79–Q83
Part 16	Final results	Q84–Q85
Part 17	Limitations and future work	Q86–Q88
Part 18	Personal reflection questions	Q89–Q93
	Glossary	A–Z

PART 1

THE ORIGINAL 14 QUESTIONS

Q1 Why did you use PyTorch instead of TensorFlow?

Q2 What is the input to the model?

Q3 What is your target (what does the model predict)?

Q4 Where does the ground truth come from?

Q5 Why predict velocity instead of position?

Q6 Which ML algorithm are you using?

Q7 Why did you choose that algorithm?

Q8 What is your loss function?

Q9 How do you prevent overfitting?

Q10 How do you split the data into train, validation and test?

Q11 What metrics do you use?

Q12 How do you test generalization?

Q13 Can the model run on a smartphone?

Q14 What happens when GNSS disappears?

Q1. Why did you use PyTorch instead of TensorFlow?

ANSWER

Both are "**deep-learning frameworks**" (software libraries that let you build and train neural networks without writing all the maths yourself). Either could have done this project. We chose PyTorch for practical reasons:

1. It runs line by line like normal Python ("**eager execution**"), so when something went wrong we could stop the program, print the numbers and see the problem immediately. This mattered a lot, because we found many data bugs (Part 4).
2. It worked directly with the laptop's NVIDIA graphics card (GPU = graphics processing unit, a chip that does many calculations in parallel and makes training much faster) through CUDA (NVIDIA's software for using the GPU).
3. It has the tools we needed for the phone version: "**quantization**" (shrinking the model's numbers from 32-bit to 8-bit) and export to a phone format (the PyTorch "**Lite interpreter**", a small runtime for phones).
4. Most recent research code for sensor-based navigation is written in PyTorch, so it was easier to compare with published work.

Honest note: PyTorch's phone runtime (PyTorch Mobile) is now deprecated (no longer updated) in favour of its successor ExecuTorch. If we build a real Android app, we would export the model to ONNX (an open, framework- neutral model format) and run it with ONNX Runtime Mobile. TensorFlow Lite would have been an equally valid choice for the phone.

Q2. What is the input to the model?

ANSWER

A 20-second “**window**” of the phone's motion sensors, sampled 10 times per second (10 Hz, “**hertz**” = times per second). So one input is 200 time steps, and at each step there are 8 numbers (“**features**”):

1-3. Linear acceleration forward, sideways and up, in the CAR's frame of reference. “**Linear acceleration**” = acceleration with gravity removed. “**Car's frame**” = rotated so that “**forward**” means the direction the car drives, not the direction the phone happens to point.

4-6. Gyroscope readings (how fast the phone is rotating around its three axes), with each phone's small constant error (“**bias**”) removed.

7. The size of the horizontal acceleration (forward and sideways combined).

8. The size of the total rotation rate.

Features 7 and 8 do not depend on how the phone is turned, which made the model a bit more robust. Before entering the model, every feature is “**normalized**” (shifted and scaled so it has average 0 and spread 1), using statistics from the training data only.

Why 20 seconds? We tested 2, 5, 10, 20 and 30-second windows; 20 s was best. A short window only shows how speed is CHANGING; a longer window shows the vibration “**signature**” of a speed (a car at 100 km/h on a motorway vibrates differently from one at 20 km/h in town).

Q3. What is your target (what does the model predict)?

ANSWER

The car's forward speed, in metres per second, at the last moment of the 20-second window. It is a “**regression**” problem (predicting a number), not a “**classification**” problem (predicting a category).

Q4. Where does the ground truth come from?

ANSWER

"**Ground truth**" = the correct answer we compare against. We used three sources:

1. GPS speed from the same phone (for training). GPS in this dataset only updates about every 9-10 seconds and holds the old value in between, so we "**interpolate**" it (draw a straight line between two GPS readings) to get a speed for every time step.
2. The car's own wheel speed (for checking). The dataset also contains the car's internal log (from its CAN bus, the car's internal computer network) with wheel-speed sensors at 10 Hz. We lined these files up with the phone files using "**cross-correlation**" (sliding one signal over the other until the speed patterns match). 39 drives lined up clearly.
3. GPS positions (for path accuracy). When we simulate a tunnel, we HIDE the real GPS positions from our system and only use them afterwards to measure how far off our estimate was.

Q5. Why predict velocity instead of position?

ANSWER

Position depends on WHERE you are in the world; speed does not. A model cannot learn from vibration that you are "**on the M5 near Cheltenham**", but it can learn that this vibration pattern means "**about 25 m/s**". Speed is a local, physical property that the sensors actually contain information about.

Once we know speed and direction, position follows by "**dead reckoning**" (starting from a known point and adding up distance travelled in a known direction, the way old sailors navigated). That is also why the project is called "**Intelligent Dead Reckoning**". Predicting speed also lets us combine the model with physics (gyroscope direction), road maps and GPS in a clean way.

Q6. Which ML algorithm are you using?

ANSWER

The main model is a one-dimensional Convolutional Neural Network ("**1-D CNN**"):

- "**Neural network**" = a mathematical function with many adjustable numbers ("**parameters**" or "**weights**") that are tuned automatically from examples.
- "**Convolutional**" = it slides small pattern detectors ("**filters**" or "**kernels**") along the time series, so the same pattern (for example, a bump or a vibration burst) is recognized wherever it appears in the window.
- "**1-D**" = it slides along one dimension, time (image CNNs slide in 2-D).

Our network (called VelocityCNNDeep) has 5 blocks. Each block has a convolution (kernel size 5), "**batch normalization**" (rescales numbers inside the network to keep training stable), "**ReLU**" (a simple rule that keeps positive numbers and sets negative ones to zero, which lets the network learn non-linear patterns) and "**max pooling**" (halves the length by keeping the largest value of each pair, so later blocks see longer stretches of time). The blocks have 64, 128, 128, 256 and 256 filters. At the end it takes the average and the maximum over time and passes them through a small fully connected layer to output one number: speed. It has about 650,000 parameters.

The full system also uses non-learned, probabilistic algorithms: an Extended Kalman Filter (Part 12) and a Hidden Markov Model for map matching (Part 10). For comparison we also trained simple models: ridge regression and gradient boosting (Q7, Part 7).

Q7. Why did you choose that algorithm?

ANSWER

Four reasons, each tested rather than assumed:

1. It beat the simple models on the same data. Test error ("**MAE**", mean absolute error = the average size of the mistake):

```
predict the average speed always ..... 8.05 m/s
ridge regression ..................... 6.51 m/s
gradient boosting .................... 5.18 m/s
our CNN .............................. 4.53 m/s
```

2. It beat other neural network sizes in "**5-fold cross-validation**" (splitting the training drives into 5 groups, training 5 times, each time testing on a different group, then averaging; this gives a more reliable comparison than a single split): the old small 2-second CNN scored 4.73 m/s, a medium CNN 3.88 m/s, our wider CNN 3.42 m/s (lower is better).
3. CNNs suit sensor time series: they detect local patterns (vibration, bumps, braking) wherever they occur.
4. It is small and fast enough for a phone: 0.71 MB after compression and under 1 millisecond per prediction.

Q8. What is your loss function?

ANSWER

The “**loss function**” is the score the network tries to make as small as possible during training; it measures how wrong the predictions are.

We used the Huber loss (in PyTorch: SmoothL1Loss). For small errors it behaves like squared error (MSE, mean squared error), which gives smooth, precise training. For large errors it behaves like absolute error, which does not explode. This matters because our labels come from GPS, which sometimes has glitches: with plain squared error, one wrong GPS speed of 50 m/s would dominate the training. Huber loss limits the damage from such outliers (“**outlier**” = a value far from the rest, often an error).

Q9. How do you prevent overfitting?

ANSWER

"**Overfitting**" = the model memorizes the training examples (including their noise) instead of learning the general pattern, so it looks great on training data and fails on new data. We used seven protections:

1. Split by whole drive. A drive is never partly in training and partly in testing; otherwise the model could "**recognize**" the drive instead of learning speed. (Windows from the same drive overlap and look alike.)
2. Early stopping. We watch the error on validation drives after every pass over the training data ("**epoch**") and keep the best version. Training stopped after 16 epochs; the best was epoch 6 (patience = 10 epochs without improvement).
3. Weight decay (L2 regularization) = a small penalty that keeps the network's numbers from growing large (value 0.0001).
4. Dropout (20%) = randomly switching off 20% of the connections in the last layer during training, so the network cannot rely on any single connection.
5. Batch normalization, which also has a mild regularizing effect.
6. Normalization statistics computed from training data only.
7. Model choices made with cross-validation and validation drives; the 9 test drives were frozen in a file (data/split.json) and used only for final reporting.

Q10. How do you split the data into train, validation and test?

ANSWER

By whole recording (drive), with a fixed random seed (so the split is the same every time), saved in data/split.json and never changed after that:

- Training: 54 drives, 109,893 windows (the model learns from these)
- Validation: 10 drives, 29,925 windows (used to choose settings and for early stopping)
- Test: 9 drives, 32,165 windows (used only to report final results) Windows are taken every 1 second ("**stride**" = how far the window moves each time), so neighbouring windows overlap by 19 seconds. Parts of one long recording (S-S3a/b/c) are always kept together.

Q11. What metrics do you use?

ANSWER

A “**metric**” is a number that measures performance. We used:

- Speed: MAE (mean absolute error, m/s) and RMSE (root mean squared error, which punishes large mistakes more).
- Distance in an outage: how far off the distance travelled is (m and % of the true distance).
- Final position: distance between our estimated position and the real GPS position when GPS returns (m). We report the median (the middle value, which is not distorted by a few extreme cases) and the P90 (90th percentile: 90% of cases are better than this, so it describes the bad cases).
- Tunnel track: error at the hidden GPS points inside the tunnel.
- Route verification: false-alarm rate, detection rate and delay.
- Phone suitability: model file size and time per prediction.

Q12. How do you test generalization?

ANSWER

“**Generalization**” = working on data the model has never seen.

1. Test drives are different recordings, often different phones, cars, roads and even countries (UK and France).
2. Cross-validation by recording (Q7).
3. A second, independent truth: car wheel speed on 39 drives.
4. Simulated tunnels at random places in each drive, where the system may use ONLY data from before the tunnel (Part 15).
5. All settings tuned on training or validation drives; test drives used once at the end.

Q13. Can the model run on a smartphone?

ANSWER

Yes, in size and speed. We compressed it with "**int8 quantization**" (storing numbers as 8-bit integers instead of 32-bit decimals): file size went from 2.63 MB to 0.71 MB, time per prediction from 9.8 ms to 0.6 ms on one laptop CPU thread, and accuracy stayed almost the same (test MAE 4.58 vs 4.53 m/s). The whole navigation engine processes 12 minutes of driving in 12.6 seconds on the laptop (about 58 times faster than real time).

Honest limit: we have not installed it on an actual phone yet. There is no native phone app; the current app is a web demo that replays recorded drives (Part 14, Part 17).

Q14. What happens when GNSS disappears?

ANSWER

"**GNSS**" (Global Navigation Satellite System) is the general name for satellite positioning; GPS is the American one. When the phone stops getting GPS fixes for more than 12 seconds, the engine switches status from "**GNSS**" to "**DR**" (dead reckoning):

1. Speed comes from the ML model, corrected at the tunnel entrance by the last real GPS speed (the correction fades out over about 20 seconds).
2. Direction starts from the last GPS direction and is updated by the gyroscope (calibrated for that particular phone).
3. The Kalman filter combines these and keeps track of how uncertain the position is (the uncertainty grows with time).
4. If a road map is available, the path is snapped to the most likely roads (status "**DR+MAP**").
5. A route monitor checks whether the car still seems to be on the planned route.
6. When GPS comes back, the filter is corrected and the whole tunnel path is recalculated backwards (RTS smoother), so the saved track becomes much more accurate.

PART 2

THE BIG PICTURE: WHAT WE BUILT AND WHY

Q15 In one sentence, what did you build?

Q16 Why is this problem important?

Q17 What is dead reckoning, and why is it hard with a phone?

Q18 What are the main building blocks of the final system?

Q19 How was the work organized?

Q20 What tools and libraries did you use?

Q15. In one sentence, what did you build?

ANSWER

A system that keeps estimating where a car is when GPS is lost (for example in a tunnel), using only a smartphone's motion sensors, a machine- learning speed model, road maps and a Kalman filter, and that checks whether the car is still on its planned route.

Q16. Why is this problem important?

ANSWER

Tunnels, underground car parks, dense city centres (“**urban canyons**”) and multi-level flyovers block or reflect satellite signals. A navigation app then either freezes or jumps around, so drivers miss exits right after tunnels. Dedicated car navigation systems use expensive sensors (wheel-speed links, high-grade gyroscopes). Doing it with a phone alone makes it cheap and available to everyone.

Q17. What is dead reckoning, and why is it hard with a phone?

ANSWER

Dead reckoning = start from a known position and keep adding "**distance travelled in a known direction**". The classical way with motion sensors is "**strapdown inertial navigation**" (INS): integrate acceleration once to get speed and twice to get position. "**Integrate**" = add up small pieces over time.

With a phone this fails quickly, because any tiny error in acceleration is added up again and again. We measured it: on one drive, double integration of the phone's accelerometer gave errors of hundreds of kilometres over 86 minutes. That is why we replaced "**integrate acceleration**" with "**learn speed directly**".

Q18. What are the main building blocks of the final system?

ANSWER

In order of the data flow:

1. Loader: reads the phone recordings and repairs their many problems.
2. Calibration: removes gravity, sensor bias and the phone's mounting angle.
3. ML speed model (1-D CNN).
4. Anchoring: corrects the model with the last GPS speed at tunnel entry.
5. Gyroscope heading: direction of travel.
6. Map matching (HMM on OpenStreetMap roads).
7. Extended Kalman Filter: fuses everything and GPS.
8. RTS smoother: corrects the tunnel track when GPS returns.
9. Route monitor: on route / off route.
10. Edge engine + API + web app: runs all of it sample by sample and shows it.

Q19. How was the work organized?

ANSWER

In 7 phases over a 15-day plan:

1. Setup and understanding the dataset (Days 1-2)
2. Calibration and preprocessing (Days 3-5)
3. AI speed estimation (Days 6-8)
4. Core dead reckoning and benchmark (Days 9-10)
5. Map matching (Days 11-12)
6. GNSS + INS fusion (Days 13-14)
7. Edge engine, app and packaging (Day 15) In practice there was also a large “**audit**” (a full re-check of all earlier work on the real data) on Day 8, where many hidden problems were found (Part 4). PROGRESS.md records every day, including failures.

Q20. What tools and libraries did you use?

ANSWER

Python with: NumPy (fast number arrays), pandas (tables), SciPy (scientific functions, signal filtering), scikit-learn (classical ML models), PyTorch (neural networks), osmnx, networkx, shapely, geopandas and pyosmium (maps, road graphs and geometry), pytest (automatic tests). For the app: HTML/CSS/JavaScript with Leaflet (an open-source map library) and OpenStreetMap map tiles. Hardware: a laptop with an NVIDIA RTX 3050 GPU.

PART 3

THE DATA

Q21 Which dataset did you use?

Q22 What sensors are in the phone recordings?

Q23 What is in the vehicle (car) files, and how did you use them?

Q24 How much data did you finally use?

Q25 Why did you not use the "Synchronised" folder for training?

Q21. Which dataset did you use?

ANSWER

IO-VNBD (Inertial and Odometry Vehicle Navigation Benchmark Dataset), published by researchers at Coventry University (UK). It contains real car drives with a smartphone logging its sensors and, for many drives, the car's own sensor log. We used 97 smartphone recordings ("**S-*.csv**" files) from the UK and France, from about 1 minute to 3.5 hours long.

Q22. What sensors are in the phone recordings?

ANSWER

Accelerometer (acceleration in 3 directions), “**gravity**” (the phone's own estimate of the gravity direction), gyroscope (rotation rate in 3 axes), magnetometer (compass), the phone's fused orientation, and GPS (latitude, longitude, altitude, speed, accuracy, direction of travel = “**course**”, number of satellites). The phone sensors are logged about 10 times per second.

Q23. What is in the vehicle (car) files, and how did you use them?

ANSWER

The car's own 10 Hz log: wheel speeds, a high-quality “**VBOX**” GPS velocity, speedometer, yaw rate and more. We used wheel speed as an independent speed truth to double-check our results (39 drives lined up clearly; 2 did not match at all and were not used).

Q24. How much data did you finally use?

ANSWER

75 of the 97 drives loaded cleanly. 14 were too short (under a minute) or never moved, and 8 recorded at 2 Hz instead of 10 Hz, so they could not be mixed with the others. From the 73 drives at 10 Hz we built about 172,000 twenty-second windows (train + validation + test).

Q25. Why did you not use the "Synchronised" folder for training?

ANSWER

The dataset comes in two versions. The “**Unsynchronised**” folder has all 97 phone files. The “**Synchronised**” folder has 72 of the same drives, cut to the time window of their matching car files. We trained on the 97 full files (more data) and used the synchronised car files only for wheel-speed truth.

PART 4

PROBLEMS IN THE DATA AND HOW WE FIXED THEM

Q26 What was the biggest surprise in the data?

Q27 What other data problems did you find?

Q28 Was there any mistake in your own process, not just in the data?

Q29 How did you make sure the fixes were real and not guesses?

Q26. What was the biggest surprise in the data?

ANSWER

The GPS speed column. Its header says “**km/h**”, but 94 of the 97 files actually store metres per second. We discovered it because our distance calculations were 3.6 times too small. We proved it by comparing two ways of measuring a drive's length: adding up speed x time, and adding up the distances between GPS positions (using the “**haversine**” formula, which computes distances on the curved Earth). With m/s they agreed (for example 38.5 km vs 37.2 km); with km/h they did not. The loader now detects the unit for each file automatically. Three files really are in km/h.

Q27. What other data problems did you find?

ANSWER

Many. Each was confirmed on real data before fixing:

1. GPS updates only every ~10 seconds and repeats the old value in between → we interpolate between real updates ("**fixes**").
2. Some files have 18 columns instead of 24 (no compass) → the loader detects both formats.
3. Some files have a trailing comma that creates an empty extra column → removed.
4. In one file the columns are shifted by one position (the date ended up in the acceleration column) → repaired; our first repair was itself off by one and was fixed later.
5. Some numbers were stored as text → converted.
6. Logger duplicates: some files contain tens of thousands of frozen, repeated rows (70,158 in one file) → removed.
7. The time counter resets to zero in the middle of some files → unwrapped so time always increases.
8. A divide-by-zero in the mounting-angle code when two rows had the same time stamp → fixed with a safe difference.
9. An integration bug in the first baseline (the first time step was counted twice) → fixed and earlier numbers recomputed.

Q28. Was there any mistake in your own process, not just in the data?

ANSWER

Yes, and we report it. Early on, the "**trained model**" file in the results folder had been produced by a demo on synthetic data (a computer-generated 2-minute drive), not by training on the real drives. We found this during the Day 8 audit, renamed it to "**velocity_model_synthetic.pt**" so nobody would use it by accident, and trained the real model properly. Also, the first train/validation split simply took the last 20% of files by name (which were all from one group of drives) and had no test set at all; we replaced it with a proper, seeded, drive-level train/validation/test split.

Q29. How did you make sure the fixes were real and not guesses?

ANSWER

Every fix was checked against all 97 real files and, where possible, against an independent measurement (GPS track length, car wheel speed). We also wrote unit tests (small automatic checks) for the loader logic, using synthetic data where we know the right answer, for example "if the speed column holds m/s, the unit detector must say m/s".

PART 5

CALIBRATION: MAKING PHONE SENSORS USABLE

Q30 What is calibration, and why is it needed?

Q31 What calibration steps does the system perform?

Q32 How do you know when the car is standing still?

Q33 How do you find the car's forward direction?

Q34 What about drives where the car never stops?

Q30. What is calibration, and why is it needed?

ANSWER

Calibration = measuring and removing a sensor's systematic errors and converting its readings into a useful frame. A phone in a car is mounted at some unknown angle, its sensors have small constant offsets, and the accelerometer always “**feels**” gravity. Without calibration, “**forward**” for the phone is not “**forward**” for the car.

Q31. What calibration steps does the system perform?

ANSWER

1. Gravity compensation: subtract the gravity vector from the measured acceleration to get “**linear acceleration**” (acceleration caused by the car's movement only).
2. Bias estimation: when the car is truly standing still, the sensors should read zero motion. Whatever they read then is the “**bias**” (constant offset), which we subtract afterwards.
3. Mount alignment: find the rotation that turns the phone's axes into the car's axes (forward, sideways, up). First “**leveling**” (using gravity to find “**up**”), then the forward direction.
4. Denoising: a “**low-pass filter**” (keeps slow changes, removes fast jitter) where needed.

Q32. How do you know when the car is standing still?

ANSWER

With “**ZUPT**” (Zero-velocity UPdaTe) detection. Our first detector used fixed thresholds tuned on synthetic data and found only 13% of real stops, because sensor noise differs about 20 times between phones. The final version is “**adaptive**”: it measures each phone's own noise level during stops confirmed by GPS, scales the thresholds to that phone, and is capped below the moving values so it can never fire while driving (before the cap it falsely flagged 88-97% of some drives as “**stopped**”).

Q33. How do you find the car's forward direction?

ANSWER

Our first method looked for moments of forward acceleration, but it used the held (repeated) GPS speed and was essentially random (it gave a 116° error with zero correlation to real acceleration). The final method compares two measurements of "**how the velocity changed between two GPS fixes**": one from the phone's sensors and one from GPS (both forward changes and turning). It finds the rotation that makes them agree best ("**Procrustes**" fitting = finding the best rotation that aligns two sets of vectors). On synthetic data it recovered the angle within 1.2°.

Q34. What about drives where the car never stops?

ANSWER

At first, 29 drives were thrown away because there was no stop to measure bias. Now we fall back to the average gravity direction for leveling and zero bias. That raised the usable drives from 57 to 75.

PART 6

BASELINES: THE SIMPLE METHODS WE HAD TO BEAT

Q35 What is a baseline and why do you need one?

Q36 What baselines did you use?

Q37 How did the classical inertial method perform?

Q35. What is a baseline and why do you need one?

ANSWER

A baseline is a simple, obvious method used as a reference. If the AI cannot beat it, the AI is not useful. The project brief explicitly asked for a baseline before using ML.

Q36. What baselines did you use?

ANSWER

1. Naive double integration of acceleration (to show why pure physics fails with a phone).
2. Strapdown INS with ZUPT (the classical inertial navigation method).
3. "**Hold last GPS speed and direction**": when GPS is lost, assume the car keeps its last speed and direction. Simple, but surprisingly strong.
4. For the speed model: "**always predict the average speed**", ridge regression and gradient boosting.

Q37. How did the classical inertial method perform?

ANSWER

Worse than just holding the last speed. On 57 drives with simulated GPS outages (median distance error at 30 / 60 / 120 seconds):

```
hold last speed ......... 56 / 151 / 314 m
INS + adaptive ZUPT ..... 107 / 286 / 784 m
```

The phone accelerometers are too noisy for double integration. This finding justified the ML approach.

PART 7

THE MACHINE-LEARNING SPEED MODEL

Q38 How was the model trained?

Q39 How did the model improve over time?

Q40 Why do you also report ridge regression and gradient boosting?

Q41 What limits the model's accuracy?

Q42 Did you try adapting the model to each phone?

Q38. How was the model trained?

ANSWER

With the “**Adam**” optimizer (an algorithm that adjusts the network's weights step by step to reduce the loss, adapting the step size for each weight), learning rate 0.001 (how big each adjustment step is), batches of 256 windows (the model looks at 256 examples, computes the average error, then updates), up to 50 epochs with early stopping, on the GPU.

Q39. How did the model improve over time?

ANSWER

The first real-data model (2-second window, small CNN) had a test error of 5.15 m/s and showed “**regression to the mean**” (it predicted too high when the car was stopped and too low at high speed, because 2 seconds show how speed changes, not what it is). The final model (20-second window, wider CNN, 2 extra magnitude features) reached 4.53 m/s. We also tried per-drive normalization of features, which made things worse, because the absolute vibration level itself carries speed information.

Q40. Why do you also report ridge regression and gradient boosting?

ANSWER

The project brief says “**start with simple models before trying neural networks**”. When we re-read the brief we realized we had only compared against “**predict the mean**”, so we added two classical models trained on exactly the same windows and split. “**Ridge regression**” = a linear model (weighted sum of inputs) with a penalty against large weights. “**Gradient boosting**” = many small decision trees (“**if-then**” rules), each correcting the errors of the previous ones. They used 72 hand-made “**features**” per window (averages, spreads, percentiles, jerkiness and vibration energy in three frequency bands for each of the 8 channels). Results on test: ridge 6.51 m/s, gradient boosting 5.18 m/s, CNN 4.53 m/s. So the CNN is 12% better than the best classical model. Gradient boosting was a strong runner-up.

Q41. What limits the model's accuracy?

ANSWER

The phone and car. The model reads speed from vibration, and vibration depends on the car's suspension, tyres, road surface and how the phone is mounted. On one test drive (S-T7) the model under-predicted by about 5 m/s the whole time: a smooth-riding car “**feels**” slow. Training with car wheel speed instead of GPS speed did not fix this (3.14 → 3.05 m/s), which shows the error is about the phone/car, not the labels.

Q42. Did you try adapting the model to each phone?

ANSWER

Yes (“**per-phone fine-tuning**”: a few extra training steps on that phone's own data before the tunnel). It helped a lot on S-T7 but broke another drive (S-Vw4) where the adaptation minutes did not cover the speeds driven later. Because it was not reliable, we did not adopt it. It is listed as future work, with safeguards.

PART 8

USING THE MODEL INSIDE A TUNNEL (ANCHORED ML)

Q43 What is "anchored ML"?

Q44 Why does the correction fade instead of staying forever?

Q45 How much did anchoring help?

Q43. What is "anchored ML"?

ANSWER

At the tunnel entrance we know the true speed (the last GPS speed). The model's prediction at that moment is usually a bit off. Anchored ML adds the difference (true minus predicted) to the model's predictions, and lets this correction fade away over about 20 seconds: speed(t) = model(t) + (GPS speed - model at entry) x e^(-t / 20 s) ("**e^(-t/20)**" is an exponential decay: the correction is 100% at the entrance, 37% after 20 s, 5% after 60 s.)

Q44. Why does the correction fade instead of staying forever?

ANSWER

Because the model's error is not constant. We tested this in Phase 6: the model's error in the 5 minutes before a tunnel barely predicts its error during the tunnel (correlation 0.19 on validation drives, where 1 would be a perfect relationship and 0 none). Keeping the correction forever would sometimes make things much worse. The 20-second fade was chosen from experiments.

Q45. How much did anchoring help?

ANSWER

Distance error on the test drives at 30 / 60 / 120 s:

```
hold last speed ........ 74 / 166 / 361 m
plain ML model ......... 87 / 136 / 221 m
anchored ML ............ 74 / 113 / 206 m
```

Checked against car wheel speed on 39 drives: anchored ML 47 / 88 / 176 m vs hold last speed 61 / 141 / 315 m.

PART 9

DIRECTION (HEADING) AND THE 2-D PATH

Q46 How do you know which direction the car is going in a tunnel?

Q47 What surprising problem did you find with the gyroscope?

Q48 Why not use the phone's compass?

Q49 How much did the gyroscope heading help?

Q46. How do you know which direction the car is going in a tunnel?

ANSWER

“**Heading**” = the direction of travel (degrees clockwise from north). We start with the last GPS direction and add the car's rotation measured by the gyroscope around the vertical axis (found using the gravity direction).

Q47. What surprising problem did you find with the gyroscope?

ANSWER

On most phones the gyroscope columns are not in the same order or sign as the other sensors' axes. The “**standard**” pairing was correct in only 17% of cases. So, for each phone and each tunnel, we try all 24 possibilities (6 axis orders x 4 sign patterns) and pick the one that best explains the GPS direction changes before the tunnel, together with a scale factor. The fit is used only if its correlation with GPS is at least 0.5 (true in 75% of outages); otherwise we keep the last GPS direction.

Q48. Why not use the phone's compass?

ANSWER

The magnetometer is disturbed by the car's steel body, electric motors and tunnel structures. The phone's compass-based orientation could be calibrated in only 20% of outages, so we did not use it.

Q49. How much did the gyroscope heading help?

ANSWER

With the true distance (to isolate the direction error), the end-position error at 30 / 60 / 120 s dropped from 98 / 239 / 651 m (straight line in the last GPS direction) to 59 / 157 / 447 m with the calibrated gyroscope. But even with perfect distance, a 2-minute outage still ended over 400 m off, because small direction errors grow into large sideways errors. That is why we needed road maps.

PART 10

MAP MATCHING: KEEPING THE CAR ON REAL ROADS

Q50 What is map matching?

Q51 How does your map matcher work?

Q52 How did you get the road maps, and what went wrong?

Q53 Could the map "cheat" by showing the true route?

Q54 How much did map matching help?

Q50. What is map matching?

ANSWER

Snapping an estimated path onto the road network, because a car can only drive on roads. We used OpenStreetMap (OSM, a free, community-made world map) road data.

Q51. How does your map matcher work?

ANSWER

It is a “**Hidden Markov Model**” (HMM), a standard method from a well-known 2009 paper by Newson and Krumm, adapted to dead reckoning:

- “**Hidden states**” = the possible positions on the roads (which we cannot see directly).
- Every 5 seconds we look at the dead-reckoned position and list nearby road positions as candidates.
- “**Emission probability**” = how likely a candidate is, given how close it is to our estimate and whether the road direction matches the gyroscope heading (allowed distance grows with time, because dead reckoning drifts).
- “**Transition probability**” = how likely it is to move from one candidate to the next, given that the driving distance along the roads should match the distance we think we travelled.
- The “**Viterbi algorithm**” finds the most likely sequence of road positions.
- It is “**causal**”: the position at time t uses only information up to t.
- Day 12 addition: “**turn consistency**”, the road's direction change must match the gyroscope's turn.

Q52. How did you get the road maps, and what went wrong?

ANSWER

We needed a map strip (“**corridor**”) 2 km wide around each drive. Problems and fixes:

1. Long drives cover up to 150 km, so whole rectangles would be enormous → corridors around the GPS track.
2. The public map server (Overpass API) became overloaded (“**at capacity**”) and rejected our bulk requests → we switched to downloading regional files from Geofabrik (ready-made OSM extracts per county) and building each corridor locally.
3. When we downloaded several files in parallel with the “**curl**” tool, four “**files**” were actually web redirect pages (“**307 moved**”) instead of maps → re-downloaded with redirect-following, and the loader now checks that each file really is a map file.
4. Old download processes from an earlier session kept running and mixed into the logs → found and stopped.
5. French regions are large files (230-370 MB) → first skipped, then completed. Final result: maps for all 87 usable drives. We checked that a Geofabrik-built map matched the server-built one (576 vs 568 road junctions for the same area).

Q53. Could the map "cheat" by showing the true route?

ANSWER

This was a real risk. The map corridor is cut around the whole drive, including the part inside the simulated tunnel. A narrow corridor would only contain the roads the car actually used. So we made it wide (2 km), and the matcher checks for every step that its whole search area lies well inside the corridor; if not, the map is not used for that outage. The map was usable in 99% / 98% / 88% of the 30 / 60 / 120-second outages.

Q54. How much did map matching help?

ANSWER

With the true distance, it brought most 30-60 second outages within GPS noise: 59 / 157 / 447 m → 9 / 33 / 306 m. With the ML speed (the real system), on the test drives: 94 / 208 / 747 m → 72 / 106 / 712 m. The map fixes direction very well, but cannot fix a wrong distance.

PART 11

IDEAS THAT DID NOT WORK (AND WHY WE KEPT THEM IN THE REPORT)

Q55 Why report failed ideas?

Q56 What was the "NHC" idea, and why did it fail?

Q57 What was "turn anchoring"?

Q58 What about learning the speed bias while GPS is available?

Q55. Why report failed ideas?

ANSWER

Because they are real results. They show the final design was tested against alternatives, and they save the next person from repeating them.

Q56. What was the "NHC" idea, and why did it fail?

ANSWER

NHC = Non-Holonomic Constraint: a car cannot move sideways, only along its heading. Our dead reckoning already obeys this (we move along the heading). We also tested an extra use: in a turn, sideways acceleration = speed x turning rate, so speed = sideways acceleration / turning rate. On 15 drives this "**turn speed**" was off by 3-17 m/s, worse than the ML model (3-4 m/s), because the phone's sideways acceleration is small and buried in vibration. Not used.

Q57. What was "turn anchoring"?

ANSWER

The idea: when the car completes a turn at a junction, the map tells us exactly where it is along the road, so we could restart distance counting there. It only applied in 12-23% of outages (few clean turns), helped only when speed was biased and hurt when speed was right. Not adopted.

Q58. What about learning the speed bias while GPS is available?

ANSWER

Explained in Q44: the bias drifts within minutes (correlation 0.19), and correcting with it did not help at 120 s (142 → 143 m) and badly hurt one drive (75 → 425 m). So the filter keeps only the short, fading correction.

PART 12

GNSS + INS FUSION (THE KALMAN FILTER)

Q59 What is a Kalman filter, in simple words?

Q60 What does your filter keep track of?

Q61 What is gating, and why do you need it?

Q62 What went wrong when you first built the filter?

Q63 What is the RTS smoother?

Q64 How did you tune the filter without cheating?

Q65 What did fusion achieve?

Q59. What is a Kalman filter, in simple words?

ANSWER

A Kalman filter is a method for combining uncertain measurements with a model of how things move. It keeps a best guess (the “**state**”) and how sure it is (the “**covariance**”, a measure of uncertainty). At every step it:

1. Predicts: "if the car drove at this speed in this direction for 0.1 s, it is now here", and increases the uncertainty a little.
2. Updates: when a measurement arrives (ML speed, GPS position), it moves the guess towards the measurement, more strongly when the measurement is reliable and the guess is uncertain. An “**Extended**” Kalman filter (EKF) handles curved, non-linear relationships (like turning) by approximating them as straight lines locally.

Q60. What does your filter keep track of?

ANSWER

Six numbers (the “**state**”): east position, north position, heading, speed, the ML model's speed error (“**ML bias**”) and the gyroscope's remaining error (“**gyro bias**”). It runs 10 times per second before, during and after the tunnel. It uses the phone's own reported GPS accuracy to decide how much to trust each GPS fix (“**adaptive noise**”).

Q61. What is gating, and why do you need it?

ANSWER

GPS sometimes gives bad positions, especially near tunnel exits, where signals bounce off walls ("**multipath**"). "**Gating**" rejects a measurement that is statistically impossible given the current estimate. We use a "**chi-square gate**" (a statistical test: if the difference between measurement and prediction, scaled by the uncertainty, is larger than a 99.9% threshold, it is treated as an outlier). In a test where 15% of GPS fixes were replaced by 200 m jumps, the gate rejected 1,143 of 1,147 bad fixes and halved the bad-case error (P90 97 m vs 198 m).

Q62. What went wrong when you first built the filter?

ANSWER

A “**death spiral**”. GPS fixes come only every ~9 seconds. In a sharp turn, the gyroscope direction could be 20-30° off by the next fix. The filter was over-confident, so it rejected the correct GPS fix as an “**outlier**”, drifted further, rejected the next one too, and never recovered (219 rejections on one drive). Two fixes:

1. The filter becomes more uncertain about direction while turning (in proportion to the turn rate).
2. If a fix is rejected right after another rejection, the filter accepts it anyway, with reduced weight: two disagreements in a row mean the filter, not the GPS, is wrong.

Q63. What is the RTS smoother?

ANSWER

RTS = Rauch-Tung-Striebel smoother. A Kalman filter only uses the past. When GPS returns at the tunnel exit, we know where the car ended up, so the RTS smoother goes backwards through the tunnel and corrects every earlier position using that knowledge. The live position is not changed after the fact, but the saved tunnel track becomes much more accurate: at the hidden GPS points inside a 2-minute tunnel, the error drops from 118 m (live) to 45 m (batch evaluation) on the test drives. This is what “**seamless handoff**” means in our project.

Q64. How did you tune the filter without cheating?

ANSWER

The filter's noise settings were tuned on the 6 validation drives only (6 settings tried). The best: a larger gyroscope scale uncertainty (0.4) and a tighter GPS direction noise (4°). The test drives were used once, for the final numbers.

Q65. What did fusion achieve?

ANSWER

On the test drives, with map matching, the end error at 30 / 60 / 120 s went from 72 / 110 / 813 m (Day 12 method, recomputed in the same run) to 66 / 112 / 484 m: a 40% improvement for long outages, mainly because the filter enters the tunnel with a better direction estimate. On the larger test + validation set the 120 s improvement was 18%.

PART 13

MAKING IT RUN ON A PHONE

Q66 What is quantization?

Q67 What were the results of compression?

Q68 Was there any problem during compression?

Q69 What exactly does the phone model expect as input?

Q66. What is quantization?

ANSWER

Neural network weights are normally 32-bit decimal numbers (“**float32**”). Quantization stores them as 8-bit integers (“**int8**”), which makes the file about 4 times smaller and the calculations much faster, at a small cost in precision. We used “**post-training static quantization**”: after training, we run 2,000 training windows through the network to measure the typical range of every internal value (“**calibration data**”), then convert.

Q67. What were the results of compression?

ANSWER

	size	test error	time per prediction
original model	2.63 MB	4.53 m/s	9.8 ms
int8 phone model	0.71 MB	4.58 m/s	0.6 ms

(time measured on one laptop CPU thread). The int8 prediction differs from the original by 0.18 m/s on average, 99% within 1.3 m/s.

Q68. Was there any problem during compression?

ANSWER

Yes. The standard “**optimize for mobile**” step stored every int8 weight twice, making the file 1.32 MB instead of 0.71 MB. We found it by looking inside the file and exported the int8 model without that step.

Q69. What exactly does the phone model expect as input?

ANSWER

The raw 20-second window of the 8 features (in a fixed order). The normalization is built into the model file, so an app cannot accidentally use the wrong statistics.

PART 14

THE EDGE ENGINE, THE APP AND ROUTE VERIFICATION

Q70 What is the "edge engine"?

Q71 Was the engine fast enough?

Q72 Does the engine give the same results as the earlier experiments?

Q73 What is route verification, and how does it work?

Q74 How did you test route verification?

Q75 What problems did you hit with route verification?

Q76 What is the app?

Q77 What went wrong with the app?

Q78 Is the app the final product?

Q70. What is the "edge engine"?

ANSWER

“**Edge**” computing = running on the device itself (the phone) rather than on a remote server, which matters in a tunnel with no internet. Our engine is one program object that takes sensor samples one at a time, exactly as a phone delivers them, and returns position, heading, speed, uncertainty and a status (GNSS / DR / DR+MAP) for every sample. Inside it: rolling calibration (last 10 minutes, refreshed every minute while GPS is available), the int8 model once per second, the Kalman filter, map matching every 5 seconds, the RTS correction when GPS returns, and the route monitor.

Q71. Was the engine fast enough?

ANSWER

At first, no: 49 seconds to process 12 minutes of driving. We “**profiled**” it (measured where the time goes) and found that converting data with pandas for every model call took half the time. We rewrote that part in plain NumPy (and proved the results were identical), ran map matching every 5 instead of 2 seconds, and used one CPU thread for the model. Final: 12.6 seconds for 12 minutes of driving, about 58 times faster than real time.

Q72. Does the engine give the same results as the earlier experiments?

ANSWER

Similar, not identical, by design. The engine is built like a real phone: it calibrates from the last 10 minutes instead of everything before the tunnel, and its filter warms up longer. On the test drives the deployed engine (int8 + map) scored 69 / 130 / 379 m at 30 / 60 / 120 s, against 66 / 112 / 484 m for the batch experiment. When we restructured the filter for the engine, we checked that the filter itself gave bit-for-bit identical outputs.

Q73. What is route verification, and how does it work?

ANSWER

The brief asked to detect whether the vehicle is following the planned route or deviating from it. Our “**route monitor**” measures the distance from the estimated position to the planned route. If that distance is larger than 40 m or 2 x the current uncertainty (whichever is larger) for 10 seconds, it switches to “**OFF ROUTE**”; it switches back the same way. Using the uncertainty stops it from raising alarms just because dead reckoning has drifted.

Q74. How did you test route verification?

ANSWER

For each simulated tunnel we gave the monitor two planned routes:

1. The route the car actually drove: any alarm is a FALSE alarm.
2. A “**wrong turn**” route, built from the road map as the shortest path that avoids the roads actually driven. The car really leaves it, so an alarm is a correct detection, but only if it comes after the car actually left the route (an earlier alarm is “**premature**” and not counted). Final test results: false alarms in 5 of 60 tunnels (8%), wrong turns detected in 35 of 54 (65%), median 7.8 seconds after the car left the route.

Q75. What problems did you hit with route verification?

ANSWER

Two:

1. The first rule (40 m, stricter when map-matched, 3-second hold) raised false alarms in 60% of test tunnels, so its detections meant nothing. We tuned a new rule on validation drives only (120 combinations tried).
2. A bug in our own test: the "**wrong**" route covered only the tunnel part, so before the tunnel the car was already far from it and the monitor was "**OFF**" before the tunnel even began. We added the pre-tunnel part of the drive to that planned route.

Q76. What is the app?

ANSWER

A mobile-friendly web page (in the mobile_app folder) served by a small Python web server (our "**API**", Application Programming Interface = a set of web addresses that programs can call to get data). You choose a drive, where the "**tunnel**" starts, its length and a correct or wrong planned route. The engine replays the real recording with GPS hidden, and the map shows the true path, the live estimate (coloured by status), the uncertainty circle, speed, heading, route status, the corrected track at the exit and the final error. Start it with "**python main.py app**" and open http://localhost:8000.

Q77. What went wrong with the app?

ANSWER

When first opened it stayed on “**loading**” forever. Two causes:

1. A style-sheet (CSS) mistake: the loading overlay's display rule overrode the browser's “**hidden**” setting, so the overlay never disappeared.
2. The first request (the list of drives) loaded every drive and its road map before answering, which took minutes. Fixes: a CSS rule that always respects “**hidden**”, and a saved list of drives that answers in about 0.1 seconds. We then verified the page by rendering it in Microsoft Edge in “**headless**” mode (a browser without a window that can save screenshots) and checking a full 2-minute replay.

Q78. Is the app the final product?

ANSWER

It is a demonstration tool, not a navigation app you install and drive with. It replays recorded drives on a laptop. The core deliverable is the tested navigation system and its phone-ready model. A real Android app is future work (Part 17).

PART 15

HONEST EVALUATION: HOW WE AVOIDED CHEATING

Q79 What is "data leakage", and how did you guard against it?

Q80 How do you simulate a tunnel?

Q81 Why median and P90 rather than averages?

Q82 Why is the test set so small for the full-path results?

Q83 How do you know the code is correct?

Q79. What is "data leakage", and how did you guard against it?

ANSWER

Data leakage = information from the answer accidentally reaching the system, which makes results look better than they would be in reality. We found and fixed several leaks:

1. Calibration: at first, calibration used the whole drive, including the future. Now each tunnel uses only data from before it.
2. Entry speed: at first, the speed at the tunnel entrance came from the interpolated GPS speed, which leans towards the NEXT GPS fix, already inside the tunnel. Now we use the last real GPS fix.
3. Map corridor: guarded as explained in Q53.
4. Split: whole drives, test drives frozen.
5. Tuning: always on training or validation drives; test drives used once.

Q80. How do you simulate a tunnel?

ANSWER

We pick random moments in a drive when the car is moving (above 3 m/s) and at least 2 minutes into the drive, right after a real GPS fix. From there we hide GPS for 30, 60 or 120 seconds. The system uses only what a phone would know at that moment. When the hidden period ends, we compare our estimated position with the first real GPS fix. The same random tunnels (fixed random seed) are used for every method from Day 10 onwards, so comparisons are fair.

Q81. Why median and P90 rather than averages?

ANSWER

A few outages go very wrong (for example, the map matcher picks the wrong road), and an average would be dominated by them. The median shows the typical case; the P90 shows how bad the bad cases are. We report both, plus averages in the result files.

Q82. Why is the test set so small for the full-path results?

ANSWER

Only 4 of the 9 test drives are long enough to have 2 minutes of driving before a 2-minute tunnel; the other 5 are 1-3 minutes long. Changing the frozen split to get more test drives would be a form of cheating, so we also report a larger "**test + validation**" table (10 drives, 150 tunnels) next to it.

Q83. How do you know the code is correct?

ANSWER

Three layers:

1. 27 automatic unit tests (pytest) that check each component on synthetic data where the right answer is known (unit detection, windows, heading fit, map filter, map matcher at a T-junction, Kalman filter, smoother, engine status changes, route monitor, model export).
2. Consistency checks when restructuring code (outputs identical to before).
3. Benchmarks on real drives, with results compared across methods.

PART 16

FINAL RESULTS

Q84 What are the headline results?

Q85 What is the single most important finding?

Q84. What are the headline results?

ANSWER

Median position error when GPS returns, on unseen test drives:

GPS lost for:	30 s	60 s	120 s
Keep last GPS speed and direction	120 m	332 m	929 m
ML speed + gyroscope heading	94 m	208 m	747 m
+ road map matching	72 m	106 m	712 m
+ Kalman filter fusion	66 m	112 m	484 m
Final phone engine (int8 model)	69 m	130 m	379 m

Other results:

- Speed model error: 4.53 m/s (about 16 km/h) on test drives.
- Tunnel track corrected when GPS returns: about 43 m error for a 2-minute tunnel (engine).
- Route verification: 8% false alarms, 65% of wrong turns detected.
- Phone model: 0.71 MB, 0.6 ms per prediction.

Q85. What is the single most important finding?

ANSWER

Each added component helps a different kind of error: the ML model fixes speed (pure physics fails with phone sensors), the gyroscope and the map fix direction, and the Kalman filter fixes the handover between GPS and dead reckoning. What remains, especially in long tunnels, is the distance error from estimating speed from phone vibration.

PART 17

LIMITATIONS AND FUTURE WORK

Q86 What are the main limitations?

Q87 What would you do next?

Q88 How long would a real phone app take?

Q86. What are the main limitations?

ANSWER

1. Long outages: after 2 minutes the error is still about 380 m, mostly from the speed estimate.
2. Phone/car dependence: speed from vibration depends on the phone, car and mounting.
3. Few test drives for long outages (4).
4. No real tunnel test with our own phone: all tests are simulated by hiding GPS in recorded drives.
5. No native phone app yet.
6. The data's GPS updates only every ~10 seconds, which limits calibration and filtering.

Q87. What would you do next?

ANSWER

1. A logger app for our own phone first, to check whether the model works on our phone and car before building more.
2. A native Android app: export the model to ONNX, port the calibration, filter and map matcher to Kotlin, and store offline maps on the phone.
3. Better speed: per-phone adaptation with safeguards, or extra signals such as OBD-II (a plug in most cars that reports wheel speed).
4. Real tunnel drives for validation.

Q88. How long would a real phone app take?

ANSWER

Roughly 4-8 weeks for a full native Android version (logger, model and filter, offline map matching, field testing), or 3-5 days for a quick demo where the phone only sends its sensors to the laptop (which needs network access, so it would not work in a real tunnel).

PART 18

PERSONAL REFLECTION QUESTIONS

Q89 What was the hardest part?

Q90 What did you learn?

Q91 If an examiner asks "is your AI really necessary?", what do you say?

Q92 If an examiner asks "where could your results be too optimistic?", what do you say?

Q93 Which number would you defend most strongly?

Q89. What was the hardest part?

ANSWER

Trusting the data. Many problems were silent: the speed unit, repeated rows, time resets, gyroscope axis order. The code “**ran**”, but the numbers were wrong. The Day 8 audit, re-checking everything against the real files, was the turning point.

Q90. What did you learn?

ANSWER

1. Always check data against an independent measurement.
2. Build a strong simple baseline first; it keeps you honest.
3. Guard against leakage everywhere, not only in the train/test split.
4. Report what did not work.
5. Verify the final product yourself (the app bug was only found by actually rendering the page).

Q91. If an examiner asks "is your AI really necessary?", what do you say?

ANSWER

Yes, and we can show it: pure inertial physics with phone sensors was about twice as bad as simply holding the last speed (Q37), while the ML speed model, once anchored, beat holding the last speed at every outage length (Q45). Simple ML models were also worse than the CNN (Q40).

Q92. If an examiner asks "where could your results be too optimistic?", what do you say?

ANSWER

1. Simulated tunnels in recorded drives are easier than real tunnels: real tunnel exits have GPS multipath, and real tunnels may have different driving behaviour.
2. The road maps are recent OpenStreetMap data, while the drives are older; roads may have changed (this could also make results worse).
3. Only 4 test drives qualify for 2-minute outages.
4. The validation drives were used for tuning, so the “**test + validation**” numbers are slightly optimistic; the test-only numbers are not.

Q93. Which number would you defend most strongly?

ANSWER

The speed model's test error (4.53 m/s) and the comparison with the simple models, because they use the largest amount of untouched test data (32,165 windows on 9 drives) and the setup was fixed before testing.

Glossary of technical terms

Accelerometer — sensor measuring acceleration (change of speed) in 3 directions; it also feels gravity.

Adam — popular training algorithm that adjusts network weights with an adaptive step size per weight.

Anchored ML — ML speed corrected by the known GPS speed at the tunnel entrance, with the correction fading over ~20 s.

API (Application Programming Interface) — a defined way for programs to talk to each other; here, web addresses that return data.

Baseline — simple reference method that a new method must beat.

Batch / batch size — group of examples processed together in one training step (256 here).

Batch normalization — rescaling of values inside a network to keep training stable.

Bias (sensor) — constant offset in a sensor's reading.

Calibration — measuring and removing sensor errors and mounting angle.

CAN bus — the car's internal computer network, carrying data such as wheel speed.

Chi-square gate — statistical test that rejects implausible measurements.

CNN (Convolutional Neural Network) — network that slides small pattern detectors over the data; "1-D" when sliding along time.

Corridor — strip of map around a drive's route (2 km here).

Covariance — the filter's measure of how uncertain each estimated value is.

Cross-correlation — sliding one signal over another to find the best match (used to line up phone and car files).

Cross-validation (k-fold) — splitting data into k groups and testing on each in turn, for a more reliable comparison.

CUDA — NVIDIA's software for running calculations on its graphics cards.

Data leakage — information from the answer accidentally reaching the system.

Dead reckoning — estimating position from a known start plus distance and direction travelled.

Deep-learning framework — software library for building neural networks (PyTorch, TensorFlow).

Dropout — randomly switching off connections during training to prevent overfitting.

Early stopping — stopping training when validation error stops improving.

Edge computing — running on the device itself instead of a remote server.

EKF (Extended Kalman Filter) — Kalman filter for non-linear (curved) relationships.

Emission / transition probability — in an HMM: how well a hidden state explains a measurement / how likely one state follows another.

Epoch — one full pass over the training data.

Feature — one input value given to a model.

GNSS — Global Navigation Satellite System (GPS, Galileo, GLONASS, ...).

GPS fix — one real position update from the GPS receiver.

GPU — graphics processing unit, used to speed up training.

Glossary of technical terms

Gradient boosting — many small decision trees, each correcting the previous ones.

Ground truth — the correct answer used for comparison.

Gyroscope — sensor measuring rotation rate.

Haversine formula — distance between two latitude/longitude points on a sphere.

Heading — direction of travel, degrees clockwise from north.

HMM (Hidden Markov Model) — model of a sequence of hidden states observed through noisy measurements; used here for map matching.

Huber loss (Smooth L1) — loss that is squared for small errors and linear for large ones, robust to outliers.

Hz (hertz) — times per second.

INS (Inertial Navigation System) / strapdown INS — navigation by integrating accelerometer and gyroscope readings.

int8 / float32 — 8-bit integer / 32-bit decimal number formats.

Integrate — add up small pieces over time (speed x time = distance).

Interpolation — estimating values between known points (straight line here).

Kalman filter — method combining a motion model with uncertain measurements.

Latency — time taken for one prediction.

Leaflet — open-source JavaScript map library used in the app.

Learning rate — size of each training adjustment step.

Linear acceleration — acceleration with gravity removed.

Loss function — score of how wrong predictions are; minimized in training.

Low-pass filter — removes fast jitter, keeps slow changes.

MAE (mean absolute error) — average size of the error.

Magnetometer — compass sensor.

Map matching — snapping an estimated path onto roads.

Max pooling — keeping the largest value in each small group, shortening the sequence.

Median — middle value of a sorted list.

Metric — number that measures performance.

Mount alignment — rotation from the phone's axes to the car's axes.

Multipath — GPS signals bouncing off walls, causing wrong positions.

NHC (Non-Holonomic Constraint) — a car cannot move sideways.

Normalization — shifting/scaling values to average 0 and spread 1.

ONNX — open, framework-neutral format for neural network models.

OpenStreetMap (OSM) — free community-made world map.

Outlier — value far from the rest, often an error.

Overfitting — memorizing training data instead of learning general patterns.

Overpass API / Geofabrik — a public OSM query server / a site offering ready-made regional OSM files.

P90 (90th percentile) — 90% of values are below this.

Parameter / weight — an adjustable number inside a model.

Glossary of technical terms

Procrustes fitting — finding the rotation that best aligns two sets of vectors.

Profiling — measuring where a program spends its time.

Quantization — storing model numbers with fewer bits (int8).

Regression — predicting a number (as opposed to a category).

Regression to the mean — predictions pulled towards the average value.

ReLU — keeps positive values, sets negative values to zero.

Ridge regression — linear model with a penalty against large weights.

RMSE (root mean squared error) — error measure that punishes large mistakes more.

Route monitor — component that decides ON ROUTE / OFF ROUTE.

RTS smoother (Rauch-Tung-Striebel) — backward pass that corrects past filter estimates using later measurements.

Seed (random seed) — fixed starting value so random choices repeat exactly.

State (filter) — the set of values the filter estimates.

Stride — how far a window moves between consecutive windows.

Test / validation / training set — data for final reporting / for choosing settings / for learning.

Unit test — small automatic check of one piece of code.

Viterbi algorithm — finds the most likely sequence of hidden states in an HMM.

Weight decay (L2 regularization) — penalty that keeps weights small.

Window — a fixed-length slice of a time series (20 s here).

ZUPT (Zero-velocity Update) — detecting moments when the vehicle is stationary, to correct sensor errors.

Thank you

Full details: README.md · PROGRESS.md · results/metrics.md

Demo: python main.py app → http://localhost:8000